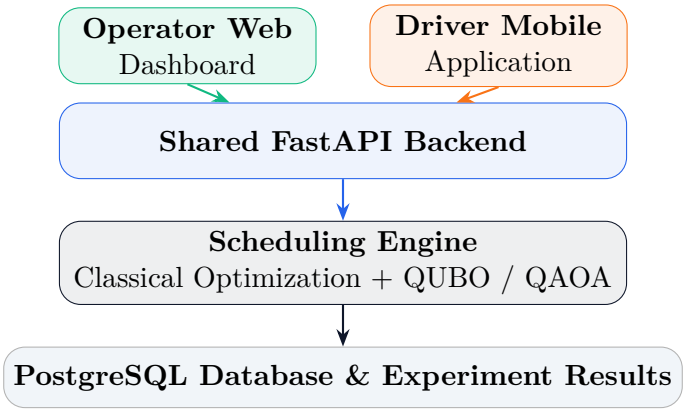
Q-CHARGE

Quantum-Assisted EV Charging Optimization Platform

Product Vision, Software Requirements, System Architecture,
Optimization Methodology, Deployment Plan, and Related Research



Contents

1 Executive Summary **3**

2 Problem Definition **3**

2.1 Background . . . 3

2.2 Problem Statement . . . 3

2.3 Project Goals . . . 3

2.4 Scope and Non-Goals . . . 4

3 Users and Product Experience **4**

3.1 Operator Web Dashboard . . . 4

3.2 EV-Driver Mobile Application . . . 4

3.3 Shared Product Principle . . . 4

4 Functional Requirements **5**

5 Non-Functional Requirements **5**

6 System Architecture **6**

6.1 Logical Architecture . . . 6

6.2 Suggested Repository Structure . . . 6

6.3 Technology Stack . . . 7

7 Data Model **7**

8 API Design **8**

9 Optimization Formulation **8**

9.1 Time Discretization and Decision Variables . . . 8

9.2 Objective Function . . . 8

9.3 Core Constraints . . . 9

9.4 Classical Baseline . . . 9

9.5 QUBO and QAOA Pathway . . . 9

10 Evaluation Plan **10**

10.1 Metrics . . . 10

10.2 Experiment Design . . . 10

11 Deployment Architecture and Plan **11**

11.1 Recommended Hosting Pattern . . . 11

11.2 Deployment Sequence . . . 11

11.3 Environment Variables . . . 11

11.4 Operational Considerations 12

12 Security, Privacy and Reliability **12**

13 Implementation Roadmap **12**

14 Testing Strategy **13**

14.1 Unit Tests 13

14.2 Optimization Tests 13

14.3 API and Integration Tests 13

14.4 Frontend and Mobile Tests 13

15 Related Open-Source Projects **13**

16 Related Research Papers **14**

16.1 How the References Inform Q-Charge 14

17 Risks and Mitigations **15**

18 Success Criteria **15**

19 Conclusion **16**

1 Executive Summary

Q-Charge is a proposed electric-vehicle (EV) charging management platform that coordinates charging across a station while helping drivers meet their battery and departure requirements. It consists of two client applications sharing one backend: an operator web dashboard and an EV-driver mobile app. Its technical core is a scheduling engine that validates charging constraints, computes a classical baseline, and investigates a quantum-assisted formulation using Quadratic Unconstrained Binary Optimization (QUBO) and the Quantum Approximate Optimization Algorithm (QAOA).

The system is designed to balance electricity cost, station peak demand, charger utilization and driver energy requirements. The first implementation should use simulated stations and synthetic data; real charger hardware and Open Charge Point Protocol (OCPP) integration can be added later. The quantum component is an experimental optimization pathway, not a claim of quantum advantage. Every quantum-assisted result must be checked for feasibility and compared with a classical method on the same instances.

The proposed product stack is React with Vite and TypeScript for the web dashboard, React Native with Expo for mobile, FastAPI for backend services, PostgreSQL for persistent storage, and Python-based optimization components using OR-Tools or SciPy alongside Qiskit. The system can be deployed as separate frontend, API, database and worker services.

2 Problem Definition

2.1 Background

As EV adoption grows, multiple vehicles may request charging at the same station during overlapping periods. If charging is unmanaged, the station can create high simultaneous power demand, incur avoidable electricity costs under time-varying tariffs, or fail to satisfy drivers' requested battery levels before departure. Scheduling is difficult because vehicles have different arrival times, departure deadlines, battery capacities, current states of charge, target states of charge and charging-rate limits.

2.2 Problem Statement

Design and implement a software platform that assigns feasible charging power to EVs over discrete time slots while respecting charger availability and station power limits. The platform should minimize a configurable combination of energy cost and peak demand, while meeting each vehicle's energy requirement when feasible. It should expose the resulting schedule to operators and drivers and provide a reproducible comparison between classical and quantum-assisted optimization.

2.3 Project Goals

- Generate feasible charging schedules for multiple EVs at a shared station.
- Reduce charging cost by shifting flexible demand to lower-price periods.
- Limit peak station demand without violating hard electrical constraints.
- Help drivers understand their recommended charging windows and estimated costs.
- Compare classical and QAOA-based approaches using the same test instances.
- Provide an extensible architecture for future renewable-energy forecasts, multiple stations and hardware integration.

2.4 Scope and Non-Goals

Included in the initial product: simulated stations, configurable EV requests, time-slot scheduling, cost and peak metrics, classical optimization, a small QUBO/QAOA experiment, operator analytics, driver schedule views and deployment-ready APIs.

Not assumed in the initial release: direct control of real chargers, payment processing, guaranteed real-time availability of physical chargers, utility-grade grid control, or demonstrated quantum advantage. These require additional integrations, testing and operational safeguards.

3 Users and Product Experience

3.1 Operator Web Dashboard

The operator dashboard is the control center for a station or a fleet of stations. Its main capabilities are:

- Station overview: number of EVs, charger status, aggregate demand and alerts.
- EV queue: arrival, departure, current and target state of charge, and charging power limits.
- Infrastructure configuration: charger count, charger power, station power limit and tariffs.
- Schedule generation: create or rerun a schedule using selected optimization methods.
- Schedule visualization: show each EV's assigned charging periods on a timeline.
- Solver comparison: compare feasibility, energy cost, peak load, utilization and runtime.
- Reporting: export scenario inputs and evaluation results for research and demonstration.

3.2 EV-Driver Mobile Application

The driver application exposes the same schedule service through a simpler, driver-focused experience:

- Create a vehicle profile with battery capacity and maximum charging rate.
- Submit current battery level, desired target level, arrival time and departure deadline.
- Select or view a compatible station.
- Receive the recommended charging window, charger assignment and estimated cost.
- View schedule status and any changes made by the operator.
- Optionally receive reminders and notifications in a later release.

The driver app should not independently calculate schedules. It should submit requests to the shared backend and display the validated result returned by the scheduling engine.

3.3 Shared Product Principle

The web dashboard and mobile app are separate user interfaces, not separate systems. Authentication, vehicle data, station data, scheduling logic, optimization jobs and results must be managed centrally by the backend. This avoids duplicated business rules and keeps the driver-visible schedule consistent with the operator's schedule.

4 Functional Requirements

ID	Requirement	Description
FR-01	Vehicle request management	Create and update EV requests including arrival, departure, battery capacity, current SoC, target SoC and maximum charging power.
FR-02	Station configuration	Configure charger count, charger power limits, station power capacity and electricity tariff by time slot.
FR-03	Feasibility validation	Reject invalid requests and detect cases where a target cannot be met within the available window.
FR-04	Classical scheduling	Generate a feasible baseline schedule using a classical solver or heuristic.
FR-05	Quantum-assisted scheduling	Convert a small, bounded scheduling instance into a QUBO and run QAOA through a simulator or supported quantum execution provider.
FR-06	Schedule validation	Independently verify charger, power, timing and energy constraints before returning any schedule as feasible.
FR-07	Metrics	Compute energy cost, peak demand, energy delivered, target completion rate, charger utilization and solver runtime.
FR-08	Operator dashboard	Display station status, EV requests, schedules and comparison metrics.
FR-09	Driver schedule view	Display the driver's assigned charging window, target level and estimated cost.
FR-10	Experiment history	Persist scenario inputs, solver settings, outputs and evaluation metrics for reproducibility.
FR-11	Authentication and roles	Separate driver and operator permissions.
FR-12	Export	Allow authorized users to export schedule and benchmark results.

5 Non-Functional Requirements

- **Correctness:** every schedule labelled feasible must pass an independent constraint validator.
- **Reliability:** solver errors, timeouts and infeasible scenarios must return clear status messages.
- **Security:** use HTTPS, validated inputs, secure password/session handling, role-based access and protected secrets.
- **Privacy:** collect only the driver data required for scheduling; use synthetic or anonymized data in demonstrations.
- **Maintainability:** separate API routes, domain services, optimizer implementations and database models.
- **Reproducibility:** record scenario parameters, random seeds where applicable, solver versions and run duration.

- **Scalability:** allow expensive optimization jobs to move from the API process to a separate worker.
- **Usability:** the operator can run a scenario and interpret the schedule and metrics without reading raw solver output.

6 System Architecture

6.1 Logical Architecture

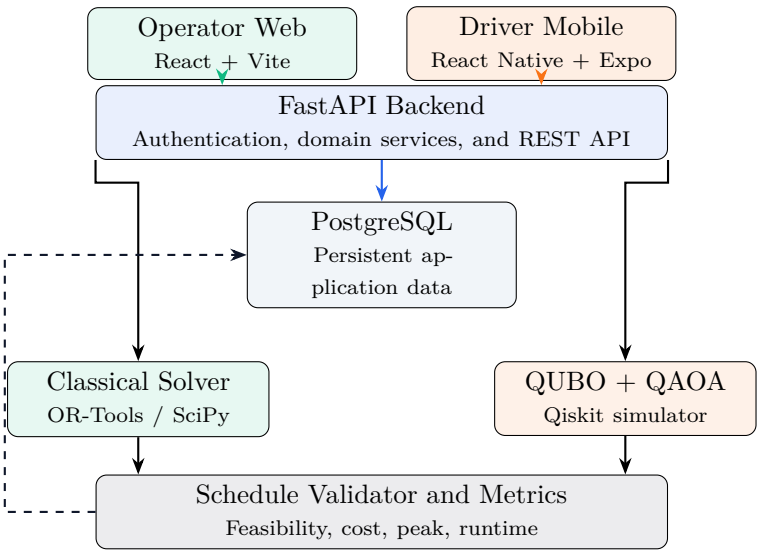


Figure 1: Q-Charge logical architecture. The API coordinates the solvers; all returned schedules pass through a common validator before persisting metrics back to the database.

6.2 Suggested Repository Structure

```
q-charge/
 apps/
    operator-web/        # React + Vite + TypeScript
    driver-mobile/       # React Native + Expo
 services/
    api/                 # FastAPI routes and domain services
    optimizer/           # Classical and QAOA implementations
    worker/              # Optional background job worker
 packages/
    shared-types/        # API types and validation contracts
 migrations/              # Alembic migrations
 tests/
    unit/
    integration/
    optimization/
 docker-compose.yml
 README.md
```

Figure 2: Q-Charge project monorepo structure.

The exact folder layout can be adjusted to match the repository tooling, but frontend code, backend code and optimization logic should remain clearly separated.

6.3 Technology Stack

Layer	Recommended technology
Operator frontend	React, Vite, TypeScript, Tailwind CSS
Driver mobile app	React Native, Expo, TypeScript
Charts	Recharts or an equivalent React chart library
Backend API	Python, FastAPI, Pydantic
Database	PostgreSQL
ORM and migrations	SQLAlchemy and Alembic
Classical optimization	OR-Tools or SciPy; use a solver appropriate to the final formulation
Quantum optimization	Qiskit and QAOA; start with small instances on a simulator
Data and analysis	NumPy, Pandas, Matplotlib
Testing	Pytest, frontend component tests and API integration tests
Deployment	Docker, managed web hosting, API hosting and managed PostgreSQL
Version control	Git and GitHub

7 Data Model

The following entities are a starting schema. They can be normalized further during implementation.

Entity	Key fields and purpose
User	id, email, password hash or identity-provider id, role, created_at. Supports driver/operator permissions.
Vehicle	id, owner_id, battery_capacity_kwh, max_charging_kw, connector_type, optional vehicle metadata.
Station	id, name, location, max_grid_power_kw, tariff configuration, station status.
Charger	id, station_id, max_power_kw, connector type, status.
ChargingRequest	id, vehicle_id, station_id, arrival time, departure deadline, current SoC, target SoC, request status.
OptimizationJob	id, station_id, method, parameters, status, created_at, started_at, completed_at, error summary.
Schedule	id, optimization_job_id, request_id, charger_id, time slot, assigned power, energy delivered.
ExperimentResult	id, optimization_job_id, feasibility, total cost, peak power, completion rate, utilization, runtime, solver metadata.

Store timestamps consistently, preferably in UTC in the database, and convert to the station's local time zone for display. Define the meaning of each time slot clearly (for example, 15-minute intervals) and use the same convention in all solvers.

8 API Design

Use a versioned REST API under `/api/v1`. The table below describes proposed endpoints, not an already deployed API.

Method	Endpoint	Purpose
GET	`/health`	Service health check.
POST	`/auth/register`	Create a user account, if local authentication is used.
POST	`/auth/login`	Authenticate a user.
GET	`/stations`	List stations visible to the current user.
POST	`/stations`	Create a station for an authorized operator.
GET	`/stations/{id}/chargers`	Retrieve charger configuration and status.
POST	`/charging-requests`	Submit a driver's charging request.
GET	`/charging-requests/{id}`	Retrieve request details and status.
POST	`/optimization-jobs`	Start a classical, QAOA or comparison run.
GET	`/optimization-jobs/{id}`	Retrieve job status and solver results.
GET	`/optimization-jobs/{id}/schedule`	Retrieve the validated schedule.
GET	`/stations/{id}/metrics`	Retrieve cost, peak, utilization and completion metrics.

For long-running jobs, return a job identifier and status such as `queued`, `running`, `completed`, `failed` or `infeasible`. The clients can poll the job endpoint initially; server-sent events or WebSockets can be considered later if live updates are necessary.

9 Optimization Formulation

9.1 Time Discretization and Decision Variables

Divide the scheduling horizon into slots indexed by $t \in T$. Let $i \in V$ represent an EV. A simple binary formulation uses

$$x_{i,t} = \begin{cases} 1 & \text{if EV } i \text{ charges in slot } t, \\ 0 & \text{otherwise.} \end{cases}$$

If charging power can vary, use $p_{i,t}$ as a continuous power variable in a classical mixed-integer model. A binary fixed-power model is easier to map to QUBO but may be less expressive.

Let c_t be the electricity price per kWh in slot t, Δt the slot duration in hours, $P_i^{\max}$ the maximum power for EV i, P^{grid} the station power limit, and E_i^{need} the energy required by EV i to reach its target state of charge.

9.2 Objective Function

A conceptual objective is

$$\min \quad \lambda_c \sum_{t \in T} c_t \Delta t \sum_{i \in V} p_{i,t} + \lambda_p P_{\text{peak}} + \lambda_u \Phi_{\text{user}},$$

where P_{peak} is the maximum aggregate charging power and Φ_{user} represents any explicitly defined soft penalty for unmet energy demand or preference violations. Hard safety and feasibility constraints must not be replaced solely by soft penalties. The weights $\lambda_c, \lambda_p, \lambda_u$ should be documented and tested for sensitivity.

9.3 Core Constraints

Availability:

$$p_{i,t} = 0 \quad \text{for slots outside EV } i\text{'s arrival–departure window.}$$

Charging rate:

$$0 \le p_{i,t} \le P_i^{\max}.$$

Station power limit:

$$\sum_{i \in V} p_{i,t} \le P^{\text{grid}} \quad \forall t \in T.$$

Energy requirement:

$$\sum_{t \in T} \eta_i p_{i,t} \Delta t \ge E_i^{\text{need}}$$

for every request whose target is feasible, where η_i represents the modeled charging efficiency. In the implementation, use a consistent battery-energy convention and avoid double-counting efficiency.

Charger availability: the schedule must not assign more simultaneous sessions than the number of compatible available chargers. If each charger serves at most one EV in a slot, this can be expressed with assignment variables $y_{i,j,t}$ and charger-specific constraints.

Departure deadline: energy required for a request must be delivered before its departure deadline. If the request is physically infeasible, return an explicit infeasible status or the best-effort result clearly labelled as such.

9.4 Classical Baseline

Implement and test a transparent baseline before attempting QAOA. Useful baselines include:

- First-Come, First-Served (FCFS), as a simple operational reference.
- A classical optimization model using OR-Tools or SciPy, selected to match the variable types and constraints.
- Optionally, a heuristic such as earliest-deadline-first for additional comparisons.

The exact classical model should be capable of producing a validated schedule and reporting infeasibility when the constraints cannot be met. A classical solver is not merely a fallback; it is the reference needed to evaluate the quantum experiment.

9.5 QUBO and QAOA Pathway

QUBO represents a problem using binary variables z and a quadratic objective:

$$\min_{z \in \{0,1\}^n} \quad z^\top Q z.$$

Constraints can be encoded using penalty terms, for example:

$$E_{\text{QUBO}}(z) = E_{\text{cost}}(z) + A E_{\text{capacity}}(z) + B E_{\text{energy}}(z) + C E_{\text{assignment}}(z),$$

where A, B, C are penalty weights. The mapping must be designed carefully: poor penalty weights can cause infeasible solutions to score better than feasible ones, and binary encodings can grow rapidly with the number of EVs and time slots.

QAOA uses a parameterized quantum circuit to sample candidate binary solutions. In Q-Charge, the first experiments should use small, controlled instances that can be independently checked by exhaustive enumeration or a trusted classical solver. QAOA outputs must pass through the same schedule validator as classical outputs. Report circuit size, number of variables, depth p, shot count, optimization settings, simulator or hardware backend, runtime, best objective and feasibility rate.

Important research limitation: QAOA is a quantum optimization method; it is not itself a quantum machine-learning model. If a competition track specifically requires QML, confirm that quantum optimization is eligible or add a justified learning component as a separate module. Do not claim quantum advantage unless the evaluation provides evidence for it.

10 Evaluation Plan

10.1 Metrics

Metric	Definition / purpose
Feasibility rate	Percentage of runs producing schedules that satisfy all hard constraints.
Energy cost	Sum of delivered energy multiplied by the applicable time-slot price.
Peak demand	Maximum aggregate charging power across the scheduling horizon.
Target completion rate	Fraction of EV requests meeting their target energy before departure.
Charger utilization	Fraction of available charger-time occupied by charging sessions.
Runtime	Wall-clock time to produce and validate a schedule, including solver setup where appropriate.
Objective quality	Objective value computed consistently for each valid schedule.
Robustness	Results across different seeds, tariffs, arrival patterns and scenario sizes.

10.2 Experiment Design

1. Create fixed benchmark scenarios with documented EV counts, chargers, time slots, tariffs and power limits.
2. Use identical inputs and objective definitions for FCFS, the classical solver and QAOA.
3. Run multiple random seeds where the algorithm is stochastic and record all solver settings.
4. Validate every candidate schedule independently before computing comparative metrics.
5. Report infeasible instances separately instead of hiding them or treating them as successful runs.
6. Compare results by instance size and report limitations, including QAOA circuit size and simulation time.

The evaluation should avoid comparing a strong classical optimizer against an intentionally weak quantum baseline. Include a transparent baseline and, where feasible, an exact solution for small instances to estimate optimality gaps.

11 Deployment Architecture and Plan

11.1 Recommended Hosting Pattern

- **Web dashboard:** deploy the React/Vite build to a static frontend host such as Vercel.
- **API:** deploy FastAPI as a Python web service on a host such as Render or Railway.
- **Database:** use managed PostgreSQL, for example Neon or a PostgreSQL offering from the API host.
- **Mobile application:** use Expo Application Services (EAS) to build installable Android/iOS artifacts. Public store release is a separate step.
- **Optimizer worker:** run expensive or long-running jobs in a separate worker process/service once needed.
- **Repository and CI:** use GitHub for version control, review and automated test/build workflows.

Provider plans, limits and prices can change. Check current documentation before choosing a production plan. The listed providers are architectural options, not endorsements or a guarantee that every plan supports every workload.

11.2 Deployment Sequence

1. Create the repository and configure development, testing and production environment variables.
2. Create PostgreSQL and run database migrations.
3. Deploy the FastAPI backend and verify its health endpoint and API documentation.
4. Deploy the operator web app and configure its public API base URL.
5. Run an end-to-end test: submit a request, run a solver, validate the schedule and display the result.
6. Configure the Expo application with the deployed API URL and build an installable test app.
7. Move heavy optimization jobs to a worker if API latency or hosting limits require it.
8. Configure logs, monitoring, backups, access controls and a repeatable deployment pipeline.

11.3 Environment Variables

Example names only; actual secret values must be stored in the hosting provider's secret manager and must never be committed to version control.

```
# Backend
DATABASE_URL=postgresql://...
SECRET_KEY=<generated-secret>
CORS_ORIGINS=https://your-dashboard-domain.example

# Web frontend (public value only)
VITE_API_URL=https://your-api-domain.example

# Mobile application
EXPO_PUBLIC_API_URL=https://your-api-domain.example
```

Variables exposed to browser or mobile bundles are not secret. Database credentials, signing keys and private tokens must remain on the server side. Restrict CORS to known origins, enforce authorization in the API, and use HTTPS in deployed environments.

11.4 Operational Considerations

A local SQLite database is suitable for early development, but a persistent managed database is preferable for a deployed multi-user service. Hosted filesystems may be ephemeral. Long-running QAOA experiments should not block ordinary API requests; create a job record and let a worker execute the optimization, then store the result for clients to retrieve.

12 Security, Privacy and Reliability

- Enforce role-based authorization for operator-only station management and driver-owned requests.
- Hash passwords using a modern password-hashing library if local authentication is used; never store plain-text passwords.
- Validate all inputs server-side, including time ranges, battery percentages, power limits and station identifiers.
- Use parameterized database access through the ORM and maintain database migrations.
- Keep credentials out of source control and rotate exposed secrets.
- Avoid exposing detailed exception traces to users; retain structured server logs without unnecessary personal information.
- Back up the database and test recovery procedures before a public production release.
- Label simulated charger status and estimated costs clearly; do not present simulated telemetry as real physical measurements.
- Ensure the optimizer cannot send commands to physical chargers unless a separate, authorized hardware-control layer has been designed and safety-tested.

13 Implementation Roadmap

Phase	Workstream	Deliverables
1	Requirements and setup	User journeys, architecture decision record, repository structure, development environment and issue backlog.
2	Simulator and data model	EV/station/charger models, synthetic scenarios, time-slot convention and input validation.
3	Classical optimizer	FCFS baseline, constrained classical solver, independent validator and unit tests.
4	Quantum experiment	Small QUBO formulation, QAOA implementation, documented penalty weights and reproducible experiments.
5	Backend API	Authentication, station/request endpoints, optimization jobs, result storage and OpenAPI documentation.
6	Operator dashboard	Station overview, EV queue, schedule timeline, solver comparison and metrics.
7	Driver app	Vehicle profile, charging request form, recommended schedule and status display.

8	Integration and hardening	End-to-end tests, error states, authorization, migrations, logs and backups.
9	Deployment and evaluation	Hosted demo, Android test build, benchmark report, documentation and presentation walkthrough.

14 Testing Strategy

14.1 Unit Tests

Test SoC-to-energy calculations, charging efficiency, tariff calculations, time-slot boundaries, charger compatibility and constraint validation.

14.2 Optimization Tests

Use small scenarios whose optimal solution can be verified by enumeration. Test impossible deadlines, insufficient station power, overlapping requests, no available chargers, zero-energy requests and varying tariffs. Confirm that invalid schedules are never marked feasible.

14.3 API and Integration Tests

Test authentication, role permissions, request validation, database persistence, job lifecycle, solver failure handling and consistency between the schedule returned to the operator and the schedule shown to the driver.

14.4 Frontend and Mobile Tests

Test loading, success, empty, invalid and error states. Verify that displayed metrics are obtained from backend results rather than hard-coded demo values. Test the mobile app on a physical device against the deployed API.

15 Related Open-Source Projects

The following projects are useful references for learning and design. They are not components that must be copied into Q-Charge. Review each repository's current license, maintenance state and attribution requirements before reusing code.

Project	Relevance	Link
SAP eMobility Smart Charging	EV fleet charging schedules, infrastructure constraints, cost and peak-load objectives, REST API. Implemented in Java/Spring Boot.	https://github.com/SAP/emobility-smart-charging
QAOA Knapsack EV Charging	Direct quantum-optimization reference for a constrained EV charging allocation problem.	https://github.com/QCL-PKNU/QAOA-Knapsack-EV-Charging
ACN Portal / ACN-Sim	Research tools, EV charging simulation, charging-session data access and algorithm comparison.	https://github.com/zach401/acnportal

SteVe	Open-source OCPP server and charging-station management concepts. Useful for future hardware integration.	https://github.com/steve-community/steve
Qiskit Optimization	Quantum optimization modeling and algorithm examples. Check current project status and IBM's recommended successor tooling before adopting.	https://github.com/qiskit-community/qiskit-optimization

16 Related Research Papers

The papers below provide background for the optimization problem and quantum formulation. Read the methods and limitations, not only the abstract. The first paper is the most directly related QAOA charging reference; the review provides broader context.

[R1] K. Kea, C. Huot, and Y. Han, "Leveraging Knapsack QAOA Approach for Optimal Electric Vehicle Charging," *IEEE Access*, vol. 11, pp. 109964–109973, 2023. DOI: 10.1109/ACCESS.2023.3320800. Official code: https://github.com/QCL-PKNU/QAOA-Knapsack-EV-Charging. A correction was published in 2025; check it when reviewing the results.

[R2] C. Si, K. T. Chau, W. Liu, and Y. Hou, "Optimal scheduling for electric vehicle charging: A review of methods, technologies, and uncertainty management," *Journal of Energy Storage*, 2025, article 117500. DOI: 10.1016/j.est.2025.117500.

[R3] A. Okada, K. Otaki, and H. Yoshida, "Joint optimization of electric vehicle routes and charging locations through learning charge constraints using QUBO solvers," *Scientific Reports*, vol. 16, article 25090, 2026. DOI: 10.1038/s41598-026-55392-1. This concerns routing and station placement rather than exactly the same station scheduling problem.

[R4] P. and S. K. Sood, "Cost-efficient energy utilization of idle electric vehicles through quantum-optimized vehicle-to-vehicle charging," *Journal of Energy Storage*, 2026, article 122534. DOI: 10.1016/j.est.2026.122534. This is a related extension involving vehicle-to-vehicle charging, not the core Q-Charge use case.

16.1 How the References Inform Q-Charge

- Use SAP Smart Charging to understand a practical charging optimizer and its API-oriented architecture.
- Use the QAOA knapsack paper and its code to study quantum encoding, penalties and experiments.
- Use ACN-Sim to learn how charging algorithms can be evaluated in repeatable scenarios.
- Use SteVe to understand OCPP and station-management concepts if real charger integration becomes a goal.
- Use the review paper to map classical optimization, uncertainty and operational challenges before claiming a research gap.

Research positioning: combining EV scheduling with QAOA is not automatically novel. A defensible contribution should be based on a specific, measured question—for example, how feasibility and solution quality change with instance size, how penalty weights affect constraint satisfaction, or how a hybrid solver performs against strong classical baselines under the same objective and test scenarios.

17 Risks and Mitigations

Risk	Impact	Mitigation
QUBO size grows quickly	QAOA circuits and simulation become impractical as variables increase.	Start with small instances, document encoding size and use classical solvers for larger cases.
Penalty weights are poorly tuned	The optimizer may prefer an infeasible schedule.	Run penalty sensitivity tests and independently validate every schedule.
No quantum advantage is observed	Quantum results may be slower or lower quality than classical results.	Report honestly; treat the project as a comparative study, not a guaranteed speedup.
Infeasible driver request	Target SoC cannot be reached by departure.	Detect infeasibility early and communicate the reason and best-effort alternatives.
API and worker timeouts	Long optimization runs can interrupt requests.	Use job records, timeouts, retries where safe and a separate worker.
Provider limits or cost changes	Hosting may not support a workload or plan as expected.	Check current limits, measure resource use and keep deployment configuration portable.
Hardware integration complexity	Real chargers require protocols, authentication and safety checks.	Begin with simulation; treat OCPP and physical control as a separate, tested integration.

18 Success Criteria

Q-Charge should be considered ready for a credible demonstration when:

- A driver can submit a valid charging request through the mobile app.
- The shared backend stores the request and starts an optimization job.
- The classical solver returns a schedule or a clear infeasibility result.
- The QAOA experiment runs on a documented small instance and its result is independently validated.
- The operator dashboard compares solver results using the same scenario and metrics.
- The driver app displays the schedule stored by the backend rather than calculating a separate one.
- The deployed application handles failures, invalid input and authorization checks without exposing secrets.
- The report presents reproducible measurements and does not claim unsupported quantum advantage.

19 Conclusion

Q-Charge is conceived as a shared-backend EV charging product with two complementary applications: an operator web dashboard and a driver mobile app. Its central engineering challenge is to produce safe, feasible and explainable charging schedules. Its research component is to investigate whether a carefully formulated QUBO/QAOA approach offers useful behavior on specific small EV scheduling instances compared with strong classical baselines.

The recommended path is to formalize the data model and constraints, implement the simulator and classical baseline, build and validate a small quantum experiment, then connect the web and mobile clients to the same backend. Deployment, benchmarking and transparent documentation should be treated as part of the product rather than as final cosmetic tasks.